

ECM2433

Ollie Mock Paper 3

The C Family

Closed Book

Additional Materials: None

Permitted Materials: No calculators permitted

Guidelines:

The duration of this paper is 2hr 00.

Answer ALL questions.

Write all answers in the answer booklet provided.

Use only black pen or pencil.

Mobile and electronic devices are not allowed.

This converted mock paper has been checked so each question is answerable and answers are not given away in the question text.

Answers and marking notes are at the end of this document.

Question 1

(a) A programmer defines `#define MULT(a, b) a * b`. (i) What is printed by `MULT(2 + 3, 4 + 5)`? Show the expansion. (ii) Give the corrected macro and explain the parentheses. (8 marks)

```
#define MULT(a, b) a * b
printf("%d\n", MULT(2 + 3, 4 + 5));
```

(b) In row-major C arrays, calculate the offset in number of integers from the start of `int A[3][4]` to `A[2][1]`. Explain how the dimensions are used. (4 marks)

```
int A[3][4];
```

(c) Write `void reverse_string(char *s)` to reverse a string in place using pointer arithmetic only. You may use `strlen`. (5 marks)

```
void reverse_string(char *s);
```

(d) Explain what a wild pointer is, contrast it with a dangling pointer, and state how Valgrind can help identify these errors. (4 marks) `*p = 10;`

```
int *p;
```

(e) Explain the difference between `fprintf(stdout, "Log\n")` and `fprintf(stderr, "Error\n")`. Why use `stderr` for errors even if both appear on the terminal by default? (4 marks)

(Total 25 marks)

Question 2

(a) Given the base classes below, derive `PetrolCar` from both using public inheritance, then write a `PetrolCar` constructor that uses a member initialiser list to initialise both base classes. (6 marks)

```
class Vehicle {
public:
    int wheels;
    Vehicle(int wheels) : wheels(wheels) {}
};
class Engine {
public:
    float power;
    Engine(float power) : power(power) {}
};
```

(b) Explain function overloading in C++. How does the compiler distinguish `void print(int)` and `void print(double)` at object-code/linker level? (4 marks)

(c) A `std::map<int, Student>` stores student records by ID. (i) Write code to check if ID 1001 exists and print the student's name. (ii) Explain why a map is often preferred over a vector for searching by ID. (7 marks)

```
std::map<int, Student> students;
```

- (d) Templates allow generic programming. (i) Define a template function `T find_max(T a, T b)` returning the larger value. (ii) What happens if it is called with two `std::string` objects, and why? (6 marks)
- (e) Explain why a class containing a `std::unique_ptr` cannot be copied by standard assignment. What must be done to transfer ownership? (2 marks)
- (Total 25 marks)**

Question 3

- (a) The function below returns the longer of two string slices. (i) Explain the lifetime error. (ii) Give the corrected signature using lifetime 'a. (5 marks)

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}
```

- (b) Rust provides `Arc<T>` and `Mutex<T>` for shared state. (i) What does `Arc` stand for, and why use it instead of `Rc` across threads? (ii) What is mutex poisoning and when does it happen? (6 marks)
- (c) Write Rust code using `iter` and `fold` to calculate the sum of all squares in a `Vec<i32>`. (5 marks)
- (d) Compare `String` and `&str` in Rust. (i) Which owns heap data? (ii) Why is `&str` usually better as a read-only function parameter? (5 marks)
- (e) What is the difference between `unwrap()` and `expect("message")` for `Option` or `Result`? When is `expect` better practice? (4 marks)
- (Total 25 marks)**

Question 4

- (a) Memory safety: (i) In C, what is a segmentation fault? Give example code. (ii) How does Rust ownership prevent use-after-free at compile time, whereas C needs runtime tools such as Valgrind? (9 marks)
- (b) Error handling: (i) Contrast C++ exceptions with Rust `Result`. Give one advantage of Rust's visibility of failures. (ii) Explain how Rust's `?` compares with manual if-error checks in C. (8 marks)
- (c) Abstraction: explain how a Rust trait bound such as `T: PartialOrd` ensures a generic function is safe to compile. (4 marks)
- (d) Resource management: describe RAI in C++. How do destructors for `std::vector` or `std::unique_ptr` prevent leaks even if a function exits via an exception? (4 marks)
- (Total 25 marks)**

Answers and marking notes

Equivalent correct code and reasoning should receive credit.

Question 1

(a) Question (8 marks): A programmer defines `#define MULT(a, b) a * b`. (i) What is printed by `MULT(2 + 3, 4 + 5)`? Show the expansion. (ii) Give the corrected macro and explain the parentheses.

Code from question:

```
#define MULT(a, b) a * b
printf("%d\n", MULT(2 + 3, 4 + 5));
```

Answer: The expansion is `2 + 3 * 4 + 5`, so normal precedence gives 19. The corrected macro must parenthesise each argument and the whole result to preserve expression grouping.

```
#define MULT(a, b) ((a) * (b))
```

(b) Question (4 marks): In row-major C arrays, calculate the offset in number of integers from the start of `int A[3][4]` to `A[2][1]`. Explain how the dimensions are used.

Code from question:

```
int A[3][4];
```

Answer: For `A[row][col]` with 4 columns, the offset is `row * 4 + col`. For `A[2][1]`, that is `2 * 4 + 1 = 9` integers from the start. Row-major order stores all of row 0, then row 1, then row 2.

(c) Question (5 marks): Write `void reverse_string(char *s)` to reverse a string in place using pointer arithmetic only. You may use `strlen`.

Code from question:

```
void reverse_string(char *s);
```

Answer: Use two pointers, one at the start and one at the last character before the null terminator. Swap the pointed characters and move the pointers inward.

```
#include <string.h>
void reverse_string(char *s) {
    char *left = s;
    char *right = s + strlen(s);
    if (right == left) {
        return;
    }
    right--;
    while (left < right) {
        char tmp = *left;
        *left = *right;
        *right = tmp;
        left++;
        right--;
    }
}
```

(d) Question (4 marks): Explain what a wild pointer is, contrast it with a dangling pointer, and state how Valgrind can help identify these errors.

Code from question:

```
int *p;
```

Answer: A wild pointer is an uninitialised pointer whose value does not point to a known valid object. Dereferencing it is undefined behaviour. A dangling pointer once pointed to a valid object, but that object has gone out of lifetime or been freed. Valgrind can report invalid reads/writes and sometimes points to the allocation/free history, but it finds errors at runtime only on executed paths.

(e) Question (4 marks): Explain the difference between `fprintf(stdout, "Log\n")` and `fprintf(stderr, "Error\n")`. Why use `stderr` for errors even if both appear on the terminal by default?

Answer: `stdout` is standard output for normal program output. `stderr` is standard error for diagnostics. They often both appear on the terminal, but they can be redirected separately. Error messages should go to `stderr` so normal output can still be piped or saved without being mixed with diagnostics.

Question 2

(a) Question (6 marks): Given the base classes below, derive `PetrolCar` from both using public inheritance, then write a `PetrolCar` constructor that uses a member initialiser list to initialise both base classes.

Code from question:

```
class Vehicle {
public:
    int wheels;
    Vehicle(int wheels) : wheels(wheels) {}
};
```

```
class Engine {
public:
    float power;
    Engine(float power) : power(power) {}
};
```

Answer: Multiple public inheritance lists both base classes after a colon. The derived constructor calls each base constructor in its member initialiser list.

```
class PetrolCar : public Vehicle, public Engine {
public:
    PetrolCar(int wheels, float power)
        : Vehicle(wheels), Engine(power) {}
};
```

(b) Question (4 marks): Explain function overloading in C++. How does the compiler distinguish void print(int) and void print(double) at object-code/linker level?

Answer: Function overloading lets multiple functions share a source name if their parameter lists differ. The compiler distinguishes them during overload resolution, then emits different mangled symbol names that encode parameter types, so the linker sees distinct functions.

(c) Question (7 marks): A `std::map<int, Student>` stores student records by ID. (i) Write code to check if ID 1001 exists and print the student's name. (ii) Explain why a map is often preferred over a vector for searching by ID.

Code from question:

```
std::map<int, Student> students;
```

Answer: Use find so the map is searched once. map is keyed by ID and gives logarithmic lookup for ordered maps, whereas a vector usually needs a linear scan unless separately indexed. auto it = students.find(1001);

```
if (it != students.end()) {
    std::cout << it->second.name << "\n";
}
```

(d) Question (6 marks): Templates allow generic programming. (i) Define a template function `T find_max(T a, T b)` returning the larger value. (ii) What happens if it is called with two `std::string` objects, and why?

Answer: The template is instantiated for the used type. For `std::string`, `operator<` or `operator>` is defined lexicographically, so the function compares the strings alphabetically.

```
template<typename T>
```

```
Answer: T find_max(T a, T b) {
    return a < b ? b : a;
}
```

(e) Question (2 marks): Explain why a class containing a `std::unique_ptr` cannot be copied by standard assignment. What must be done to transfer ownership?

Answer: `unique_ptr` represents unique ownership, so copying is disabled to prevent two objects owning the same resource. Transfer ownership with move semantics, such as `obj2 = std::move(obj1)`, after which `obj1` no longer owns the pointer.

Question 3

(a) Question (5 marks): The function below returns the longer of two string slices. (i) Explain the lifetime error. (ii) Give the corrected signature using lifetime 'a'.

Code from question:

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}
```

Answer: The return value may be borrowed from x or y, so Rust needs to know the returned borrow is tied to the input borrows. The annotation does not extend lifetimes; it describes the relationship.

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
```

```
Answer: if x.len() > y.len() { x } else { y }
}
```

(b) Question (6 marks): Rust provides `Arc<T>` and `Mutex<T>` for shared state. (i) What does Arc stand for, and why use it instead of Rc across threads? (ii) What is mutex poisoning and when does it happen?

Answer: Arc means atomically reference counted. It is safe for shared ownership across threads because its count updates are atomic; Rc is not thread-safe. A mutex becomes poisoned when a thread panics while holding the lock. Later lock attempts

```
return a PoisonError to signal the protected data may have been left in an inconsistent state.
```

(c) Question (5 marks): Write Rust code using `iter` and `fold` to calculate the sum of all squares in a `Vec<i32>`.

Answer: `fold` starts with an accumulator and updates it for each item. `iter` gives references, so the closure dereferences n.

```
let total = values
    .iter()
```

```
.fold(0, |acc, n| acc + (*n * *n));
```

(d) Question (5 marks): Compare String and &str in Rust. (i) Which owns heap data? (ii) Why is &str usually better as a read-only function parameter?

Answer: String owns growable UTF-8 text, usually heap-allocated. &str is a borrowed string slice. A read-only function should usually accept &str because it can accept borrowed String contents and string literals without taking ownership or forcing allocation.

(e) Question (4 marks): What is the difference between unwrap() and expect("message") for Option or Result? When is expect better practice?

Answer: Both extract the success value or panic on None/Err. unwrap uses a generic panic message. expect lets you provide a specific message explaining the assumption that failed, so it is better when a panic represents a violated invariant and the message will help debugging.

Question 4

(a) Question (9 marks): Memory safety: (i) In C, what is a segmentation fault? Give example code. (ii) How does Rust ownership prevent use-after-free at compile time, whereas C needs runtime tools such as Valgrind?

Answer: A segmentation fault is an OS signal commonly caused by accessing memory the process is not allowed to access, such as dereferencing NULL. In C, use-after-free compiles but is undefined behaviour. Rust tracks ownership and lifetimes, so a value cannot be used after it is moved or dropped, and references cannot outlive the data they borrow.

```
int *p = NULL;
```

Answer: `*p = 3;` /* likely segmentation fault */

(b) Question (8 marks): Error handling: (i) Contrast C++ exceptions with Rust Result. Give one advantage of Rust's visibility of failures. (ii) Explain how Rust's ? compares with manual if-error checks in C.

Answer: C++ exceptions can be thrown separately from the function return type. Rust Result<T, E> makes recoverable failure visible in the type signature, so callers must handle or propagate it. The ? operator returns early on Err and unwraps Ok, replacing repetitive manual checks while preserving the error.

(c) Question (4 marks): Abstraction: explain how a Rust trait bound such as T: PartialOrd ensures a generic function is safe to compile.

Answer: A trait bound states the operations the generic type must support. T: PartialOrd tells the compiler that comparison operations such as < or > are available for T, so generic code using those operations only compiles for suitable types.

(d) Question (4 marks): Resource management: describe RAII in C++. How do destructors for std::vector or std::unique_ptr prevent leaks even if a function exits via an exception?

Answer: RAII ties resource lifetime to object lifetime. Resources are acquired in constructors and released in destructors. During stack unwinding after an exception, local objects are destroyed, so vector frees its storage and unique_ptr deletes its owned object automatically.